

Impossibility of VDFs in the ROM: The Complete Picture

Hamza Abusalah¹, Karen Azari², Chethan Kamath³, Erkan Tairi^{4*}, and
Maximilian von Consbruch⁵

¹ IMDEA Software Institute
`hamza.abusalah@imdea.org`

² University of Vienna, Faculty of Computer Science, Vienna, Austria
`karen.azari@univie.ac.at`

³ IIT Bombay
`ckamath@cse.iitb.ac.in`

⁴ University of California, Berkeley
`erkan.tairi@berkeley.edu`

⁵ University of Vienna, UniVie Doctoral School Computer Science DoCS, Faculty of
Computer Science, Vienna, Austria
`mvconsbruch@mailbox.org`

Abstract. We study whether Verifiable Delay Functions (VDFs) can be constructed in the (plain) Random Oracle Model (ROM) without any computational assumptions. A first partial answer to this question is due to Mahmoody, Smith, and Wu [ICALP 2020] where they rule out *perfectly unique* VDFs in the ROM. Building on this result, Guan, Riazanov, and Yuan [CRYPTO 2025] prove that VDFs with *computational uniqueness* and *public-coin setup* are impossible in the ROM. These works leave open the possibility of constructing computationally unique VDFs with private-coin setup in the ROM. We resolve this problem and additionally rule out computationally-unique VDFs with *computationally expensive* private-coin setup in the ROM.

1 Introduction

A Verifiable Delay Function (VDF) [BBBF18] is a function that takes a long *sequential time* to compute, but whose output can be verified quickly. This property allows translating out (sequential) computational steps into wall-clock time in the physical world. As a result, VDFs have been shown useful in a variety of applications such as randomness beacons [LW15, BBBF18, EFKP20a, Sta20], computational time-stamping [LSS20], resource-efficient blockchains [CP19], and proofs of replication [ABBK16].

VDF constructions abound [BBBF18, Pie19, Wes19, DMPS19, DGMV20, EFKP20b, EFKP20a, LV20, BHR⁺21, JMRR21, HHK⁺22, BCH⁺22, CSRT22,

* Work done while the author was at ENS Paris.

HHKK23, LM23, CLM23, OKC+25]. Since these constructions either rely on advanced cryptographic primitives like randomized encodings, SNARGs, and FHE or require structured algebraic settings like groups of unknown order, they end up relying on structured hardness assumptions. Therefore, a fundamental question this impressive body of work leaves open is whether VDFs can be black-box constructed from one-way functions or even collision-resistant hashing.

A series of results [DGMV20, MSW20, GRY25] *partially* answer this question in the *negative* by proving various *restricted* variants of VDFs to be impossible to construct in the (Parallel) Random Oracle Model (ROM) [BR93, AS15]. In the (Parallel) ROM, algorithms have access to a random oracle that they query in parallel in a series of *rounds*. Thus, the running time of an algorithm is the number of *rounds* of queries it makes.

Defining VDFs in the ROM. Let λ be a security parameter and $f : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$ a random oracle. A VDF (in the ROM) is a tuple of oracle-aided (possibly randomized) algorithms $\Pi_{\text{VDF}} := (\text{Setup}^f, \text{Eval}^f, \text{Verify}^f)$ such that Setup^f , on input a security parameter λ and a time parameter T , probabilistically outputs public parameters pp ; Eval^f , on input pp and input x , deterministically outputs an output y and probabilistically outputs a proof π ; and Verify^f , on input pp and an input-output-proof triple (x, y, π) , deterministically outputs a bit indicating acceptance or rejection. Π_{VDF} must satisfy efficiency, completeness, sequentiality, and uniqueness:

- *Efficiency* requires that $\text{Setup}^f(\lambda, T)$ makes at most $q_s \in \text{poly}(T, \lambda)$ queries to f , $\text{Eval}^f(\text{pp}, x)$ makes at most $q_e \in \text{poly}(T, \lambda)$ queries to f in T rounds, and $\text{Verify}^f(\text{pp}, x, y, \pi)$ makes at most $q_v \in \text{poly}(\log(T), \lambda)$.
- *Completeness* requires that every honest function evaluation results in an accepting verification, i.e., $\text{Verify}^f(\text{pp}, x, y, \pi) = 1$ for every $\text{pp} \leftarrow \text{Setup}^f(\lambda, T)$ and $(y, \pi) \leftarrow \text{Eval}^f(\text{pp}, x)$.
- *Sequentiality* with gap $\epsilon \in [0, 1)$ requires that, except with negligible probability, no two-stage adversary (A_0, A_1) where A_0 , on input $\text{pp} \leftarrow \text{Setup}^f(\lambda, T)$, makes $\text{poly}(T, \lambda)$ queries to f and outputs a state st , and A_1 , on input st and a random domain input x , makes $\text{poly}(T, \lambda)$ queries to f in T^ϵ rounds and outputs y where $(y, \pi) \leftarrow \text{Eval}^f(\text{pp}, x)$.
- *Computational (non-adaptive) uniqueness (a.k.a. soundness)* requires that, except with negligible probability, no adversary that, on input $\text{pp} \leftarrow \text{Setup}^f(\lambda, T)$ and a random domain input x , makes $\text{poly}(T, \lambda)$ queries to f and finds (y', π') such that $\text{Verify}^f(\text{pp}, x, y', \pi') = 1$ and $y' \neq y$ where $(y, \pi) \leftarrow \text{Eval}^f(\text{pp}, x)$. Uniqueness is *perfect* if there exists no such triplet (x, y', π') .

Related work and open problems. Both [DGMV20, MSW20] rule out *tight* VDFs in the ROM. Informally, a VDF is tight if it can be evaluated in time T but not less than $T - T^\delta$ for any constant $\delta < 1$. Even without the (non-standard) tightness requirement, [MSW20] proves that *perfectly-unique* VDFs cannot be constructed in the ROM. Recently, this result was extended to rule out even VDFs with *computational uniqueness* in the ROM [GRY25].

However, there are a few caveats to these negative results. On the one hand, [MSW20] and [GRY25] rule out *non-tight* VDFs with “efficient” setup,

i.e., when **Setup** runs in time at most $\text{poly}(\log(T), \lambda)$, and leave open the possibility of constructing standard VDFs in the ROM when setup is “expensive”, i.e., when **Setup** runs in time $\text{poly}(T, \lambda)$. On the other hand, while [MSW20] allows for efficient *private-coin* setup, [GRY25] only considers *public-coin* setup, i.e., when **pp** also contains the random coins used by **Setup**. Furthermore, in a second result in [MSW20], *tight* VDFs with *expensive private-coin* setup are ruled out by ruling out the weaker object of *tight* Proofs of Sequential Work (PoSW) [MMV13, MSW20] with expensive private-coin setup in the ROM. A PoSW can be seen as a VDF without the uniqueness guarantee. Interestingly, the techniques of [MSW20] that rule out *tight* PoSWs/VDFs with expensive setup don’t seem to readily extend to *standard/non-tight* VDFs and the authors left this as an open problem [MSW20, Section 4.1].

Therefore, these negative results [DGMV20, MSW20, GRY25] leave open the possibility of constructing standard/non-tight VDFs in the ROM (without any computation assumptions) with *computational uniqueness and expensive private-coin setup*. We remark that this is the most general case, and hence, it is conceivable that it is easier to construct VDFs in this relaxed setting where setup is expensive and uniqueness is only computational.

1.1 Our Results

We resolve this open problem and show that *no* (even non-tight) VDFs exist in the plain ROM (i.e., without additional computational assumptions), even when considering expensive private-coin setup. We summarize the comparison between our results and prior works on standard VDFs in Table 1.

	Expensive Public-Coin Setup	Efficient Private-Coin Setup	Expensive Private-Coin Setup
Perfect Uniqueness	[MSW20], [GRY25]	[MSW20]	this work
Computational Uniqueness	[GRY25]	this work	this work

Table 1. Comparison between prior works on (non-tight) VDFs and our results.

Concretely, consider a VDF in the ROM, i.e., all algorithms have access to a random oracle f (see Definition 1 for a formal description). Assume that the **Setup**, **Eval** and **Verify** algorithms make at most q_s, q_e and q_v queries to f , respectively, where $q_e \in \text{poly}(T, \lambda)$, $q_v \in \text{poly}(\log(T), \lambda)$, and $q_s \in \text{poly}(\log(T), \lambda)$ for *efficient* setup or $q_s \in \text{poly}(T, \lambda)$ for *expensive* setup. Our results are informally stated below, and we refer to Corollary 1 for the formal statement.

Theorem 1 (Informal). *Let Π_{VDF} be any VDF (with private-coin setup) in the ROM, then at least one of the following holds:*

- *There exists an adversary A_s that breaks sequentiality of Π_{VDF} by making $O(q_s q_e q_v)$ queries in the preprocessing phase and $O(q_v)$ rounds of queries in the online phase.*
- *There exists an adversary A_u that breaks computational uniqueness of Π_{VDF} by making at most $O(q_s q_e q_v)$ queries.*

In order to prove Theorem 1, we first rederive in Section 4 the results of [GRY25], which were based on techniques from the field of query complexity (concretely, from decision tree algorithms), using the techniques developed in prior ROM separations [MMV11, MSW20, DGMV20]. We regard this reformulation of the techniques from [GRY25] as an additional contribution, as it renders the results of [GRY25] more accessible to the broader cryptographic community. Furthermore, this reformulation allows us to reconcile the techniques developed in both [MSW20] and [GRY25], and in turn extend them to the setting of standard/non-tight VDFs (with expensive private-coin setup) in Section 5.

1.2 Additional Related Work

The area of timed cryptography started with Time-Lock Puzzles (TLPs) [RSW96], which are similar to VDFs in the sense that they also provide strong sequentiality guarantees. Mahmoody et al. [MMV11] showed an impossibility result for TLPs in the ROM. The techniques developed in that work turned out to be useful also for the impossibility results on VDFs [MSW20].

A timed primitive strongly related to VDFs is Proofs of Sequential Work (PoSWs) [MMV13], which are proofs of work with sequentiality guarantees. The main difference between PoSWs and VDFs is that PoSWs do not provide any uniqueness guarantees. Not accounting for VDFs, all PoSW constructions are in the plain ROM [MMV13, CP18, AKK⁺19, DLM19, AFGK22, AC23, Abu25].

Delay functions [GS98, LW15, ARS24] are similar to VDFs as they provide sequentiality guarantees, however, unlike VDFs they lack verifiability.

Lastly, Rotem et al. [RSS20] showed that generic-group VDFs (and generic-group delay functions) cannot be constructed in cyclic groups of known order.

1.3 Organisation

We start with an overview of our techniques in Section 2. Since our work builds on [MSW20, GRY25], we warm up by discussing their results and techniques in a unified language in Section 4. In Section 5, we present our main result. Finally, the relevant definitions and notation are provided in Section 3.

2 Technical Overview

Since our work builds on [MSW20, GRY25], we warm up by discussing their results for public-coin setups and the underlying techniques in Sections 2.1 and 2.2. In Section 2.3, we describe some existing approaches that partially deal with

private-coin setups. Finally, in Section 2.4, we explain how we overcome these limitations to rule out VDFs with expensive private-coin setups. For simplicity, we focus in this technical overview on VDFs with *perfect completeness* – but the argument easily generalises to VDFs with a completeness error. We refer the reader to Sections 4 and 5 for the formal details. Let us first fix some notation and conventions pertaining to the ROM and VDFs.

- We work in the parallel random-oracle model (PROM) [BR93, AS15], where every algorithm has access to a random oracle f , which it can query *in parallel*. By “queries”, we refer to the total number of queries an algorithm makes to f . We will distinguish this complexity measure from “time”, which we will use to refer to the *total number of rounds* in which f is invoked by an algorithm, potentially involving several oracle calls in parallel per round. We refer to lazy sampling of a random oracle in the standard sense. We refer to Section 3.2 for formal definitions.
- For a VDF Π , we use q_s , q_e and q_v , respectively, to denote the maximum number of *queries* made to f by the setup, evaluation and verification algorithms. If T and λ are the time and security parameters, respectively, then by definition $q_e \in \text{poly}(T, \lambda)$ and $q_v \in \text{poly}(\log(T), \lambda)$. We say that the setup is “efficient” if $q_s \in \text{poly}(\log(T), \lambda)$, and “expensive” if $q_s \in \text{poly}(T, \lambda)$. Finally, we say the setup is “private-coin” if the random coins used to sample the public parameters are secret; otherwise, we say the setup is “public-coin”. We refer to Section 3.3 for formal definitions.

2.1 Breaking Perfectly-Unique VDFs with Public-Coin Setup

Our starting point is the work of Mahmoody, Smith and Wu [MSW20], who construct a *sequentiality breaker* for any ROM-based VDF $\Pi := (\text{Setup}, \text{Eval}, \text{Verify})$ that is perfectly-unique and has a public-coin setup. Their sequentiality breaker, denoted A_{MSW} , follows the blueprint in [IR89, Rud88, MMV11] and is described on a high level below⁶ – we refer to Algorithm 2 for a formal description. The parameter t_{on} determines A_{MSW} ’s online running time and is set, for reasons we explain in the analysis below, to $6q_v$.

Sequentiality Breaker A_{MSW}

1. *Preprocess.* On input public parameters $\text{pp} := \text{Setup}^f(\lambda, T; r_S)$, where r_S denotes the *public* random coins used for setup, initialise a partial function p_1 with the query-response pairs involved in computing pp .
2. *Learn online.* On input a challenge x , sample $i \leftarrow \{1, \dots, t_{\text{on}}\}$, and for each round $j \in \{1, \dots, i\}$:

⁶ Our presentation differs slightly from [MSW20], in which A_{MSW} carries out the “Learn online” step (Step 2) for t_{on} rounds, after which it outputs the *majority element* in $\{y'_1, \dots, y'_{t_{\text{on}}}\}$. Looking ahead, our variant results in a simpler analysis.

- (a) Lazily sample a “simulated” oracle g_j that *extends* the partial function p_j and then compute $(y'_j, \pi'_j) \leftarrow \text{Eval}^{g_j}(\text{pp}, x)$, where we say “ g_j extends p_j ” if $g_j(m) = p_j(m)$ holds for all queries m in p_j ’s domain $\text{Dom}(p_j)$.
- (b) Let $Q^* := \{m_1, \dots, m_\ell\}$ denote the set of *new* queries made to g_j during computation of (y'_j, π'_j) , where a query m is deemed to be “new” if $m \notin \text{Dom}(p_j)$. Query f on Q^* *in parallel* to obtain a response $(a_1, \dots, a_\ell)$.
- (c) Update knowledge of f by constructing a new partial function

$$p_{j+1}(m) := \begin{cases} p_j(m) & \text{if } m \in \text{Dom}(p_j) \\ a_l & \text{if } m = m_l \text{ for some } m_l \in Q^* \end{cases}$$

3. Output y'_i .

Following the argument in [MSW20], let us analyse why A_{MSW} ’s output y'_i (Step 3) constitutes a valid break of Π ’s sequentiality. Let $(y, \pi) \leftarrow \text{Eval}^f(\text{pp}, x)$, and let Q_V denote the set of queries made while computing $\text{Verify}^f(\text{pp}, x, y, \pi)$. We say that A_{MSW} “learns” in a round j if Q^* *contains at least one query* from Q_V , i.e., $Q^* \cap Q_V \neq \emptyset$. Since $|Q_V| \leq q_v$ and f is the *given* oracle subsequently queried on Q^* (Step 2b), there can be *at most* q_v rounds in which A_{MSW} learns. Now we argue that A_{MSW} breaks sequentiality in any round in which it *does not learn*! Assuming round i is one of the rounds in which A_{MSW} does not learn, it is possible to exploit *perfect uniqueness* to argue that $y'_i = y$: if, for the sake of contradiction, $y'_i \neq y$, then it can be shown that relative to the “hybrid” oracle

$$h_s(m) := \begin{cases} g_i(m) & \text{if } m \in Q^* \\ f(m) & \text{otherwise,} \end{cases} \quad (1)$$

Π ’s uniqueness is violated. To be specific, for $(y_i, \pi_i) \leftarrow \text{Eval}^{h_s}(\text{pp}, x)$ both

$$y_i = y'_i \neq y \quad \text{and} \quad (2)$$

$$\text{Verify}^{h_s}(\text{pp}, x, y, \pi) = 1 \quad (3)$$

hold, and therefore (y, π) is a valid break of uniqueness relative to h_s . This contradicts Π ’s perfect uniqueness. To see why Equation (2) holds, note that h_s and the simulated oracle g_i are consistent when it comes to *evaluation of* (pp, x) : by definition, h_s agrees with g_i on $\text{Dom}(p_i) \cup Q^*$, which contains all queries used to evaluate (pp, x) relative to g_i . As (y_i, π_i) is output by $\text{Eval}^{h_s}(\text{pp}, x)$, Equation (2) follows since the *function* output y'_i of Eval^{g_i} on public parameters pp and input x is computed *deterministically*. To see why Equation (3) holds, note that h_s and f are consistent when it comes to *verifying* (pp, x, y, π) : conditioned on A_{MSW} *not learning* in round i , Q^* does not contain any query from Q_V and thus $\text{Verify}^f(\text{pp}, x, y, \pi) = 1$ implies $\text{Verify}^{h_s}(\text{pp}, x, y, \pi) = 1$. To complete the argument, we need to set the parameter t_{on} appropriately. Specifically, we need to

set t_{on} to be sufficiently larger than q_v , so that a random round $i \leftarrow \{1, \dots, t_{on}\}$ (Step 2) is likely to be one in which A_{MSW} does not learn. For instance, for $t_{on} = 6q_v$, it can be shown that A_{MSW} does not learn in a random round i with probability at least $1/3$ (see Theorem 2).

Finally, let us see why A_{MSW} constitutes a valid sequentiality breaker. Note that the preprocessing step requires q_s queries. Moreover, for the parameters discussed above, the online learning phase (Step 2) requires at most $t_{on} = 6q_v$ time. It follows that A_{MSW} is a valid breaker since $q_v \in \text{poly}(\log(T), \lambda)$ and $q_e \in \text{poly}(T, \lambda)$. We refer the reader to Section 4.1 for more details.

2.2 Breaking Computationally-Unique VDFs with Public-Coin Setup

When Π 's uniqueness requirement is relaxed from perfect to computational, the reasoning we used above to claim that uniqueness is violated relative to h_s is no longer valid. To be specific, $\text{Verify}^{h_s}(\text{pp}, x, y, \pi) = 1$ no longer guarantees that $y'_i = y$, as π could have been a *bad proof* relative to h_s ,⁷ where by “bad proof” we mean a proof that leads to Verify accepting a wrong output. Although such bad proofs don't exist in case Π is perfectly unique, they do exist in case Π is only computationally unique. In fact, the computational uniqueness requirement for a VDF states that it is hard to find such bad proofs relative to f (see Definition 1). At this point, Guan, Riazanov and Yuan [GRY25] first observe that *in case* Equations (2) and (3) do hold but $y'_i \neq y$, A_{MSW} breaks Π 's uniqueness, albeit relative to a *wrong* oracle. In more detail, relative to h_s we have (y, π) and (y'_i, π'_i) such that $y'_i \neq y$ and $\text{Verify}^{h_s}(\text{pp}, x, y, \pi) = 1$. Then, [GRY25] make a second crucial observation: the original world (f, h_s) , with f and h_s defined as above, is distributed identically to a “flipped” world (h_u, f) , where h_u is a hybrid oracle that is closely related to h_s (to be explained soon). Whenever $y'_i \neq y$, it is therefore possible to argue that A_{MSW} breaks uniqueness relative to f in the flipped world *with the same probability* that it breaks uniqueness relative to h_s in the original world. Keeping this fact in mind, the uniqueness breaker by [GRY25], denoted A_{GRY} , is implemented as follows: for $i \leftarrow \{1, \dots, t_{on}\}$, A_{GRY} works exactly like A_{MSW} till round i . But then in round i , it flips the role of simulated oracle g_i and given oracle f , and uses f *itself* as the simulated oracle. A more detailed description is given below – we refer to Algorithm 3 for the formal description. Like we did for A_{MSW} , the parameter t_{on} can be set to $6q_v$.

Uniqueness Breaker A_{GRY}

1. *Preprocess.* On input public parameters $\text{pp} := \text{Setup}^f(\lambda, T; r_S)$, where r_S denotes the *public* random coins used for setup, initialise a partial function p_1 with the query-response pairs involved in computing pp .

⁷ Note that π is guaranteed to be a “good proof” *relative to f* since it is honestly generated relative to f and Π is *perfectly correct*. However, this guarantee does not necessarily hold relative to h_s since π is not honestly generated relative to h_s .

2. *Learn online.* Sample $i \leftarrow \{1, \dots, t_{on}\}$. For each round $j \in [i - 1]$, carry out Step 2a to Step 2c as in A_{MSW} .
3. *Construct hybrid oracle.* For round i :
 - (a) Compute $(y'_i, \pi'_i) = (y, \pi) \leftarrow \text{Eval}^f(\text{pp}, x)$, and let Q^* denote the new queries made to f during this computation.
 - (b) Lazily sample a simulated oracle g_i that *extends* p_i .
 - (c) Construct the hybrid oracle h_u as

$$h_u(m) := \begin{cases} g_i(m) & \text{if } m = m_l \text{ for some } m_l \in Q^* \\ f(m) & \text{otherwise.} \end{cases} \quad (4)$$

4. Output (y''_i, π''_i) , where $(y''_i, \pi''_i) \leftarrow \text{Eval}^{h_u}(\text{pp}, x)$.

Let us analyse why A_{GRY} 's output (y''_i, π''_i) (Step 4) constitutes a valid break of uniqueness. Assuming $y \neq y''_i$, we need to argue that $\text{Verify}^f(\text{pp}, x, y''_i, \pi''_i) = 1$. Since the original world and the flipped world can be shown to be identically distributed, we can apply the same line of argument we used for A_{MSW} to claim that A_{GRY} *also likely does not learn* in its round i if we set t_{on} large enough, say $t_{on} = 6q_v$ (see Theorem 3). To be precise, we can claim that the set Q^* (Step 3a) *contains no query* made while computing $\text{Verify}^f(\text{pp}, x, y''_i, \pi''_i)$. As a result, the oracles f and h_u are *consistent* when it comes to queries made by $\text{Verify}^f(\text{pp}, x, y''_i, \pi''_i)$ and $\text{Verify}^{h_u}(\text{pp}, x, y''_i, \pi''_i)$, which implies $\text{Verify}^f(\text{pp}, x, y''_i, \pi''_i) = 1$. Translating this informal argument into a formal proof requires careful handling of the random coins used by the adversaries and the challenger in the respective security games – we refer to Section 4.2 for the details.

Finally, note that A_{GRY} is a valid uniqueness breaker since it is query-efficient: the preprocessing step requires q_s queries and the learning stage requires at most $6q_v q_e$ queries, the total number of queries is at most $q_s + 6q_v q_e$, which is $\text{poly}(T, \lambda)$.

2.3 Handling *Private-Coin* Setups: Challenges and Existing Approaches

Recall how we used the notion of query learning to argue *consistency* of hybrid oracles in A_{MSW} (Equation (1)) and A_{GRY} (Equation (4)). The main challenge in handling a private-coin setup (efficient or expensive) is that we need to take into account queries not only in Q_V but also in Q_S , the set of queries involved in computing $\text{Setup}^f(\lambda, T; r_S)$. We were able to avoid this issue when the setup was public-coin since the coins r_S it uses are available, and can be used to initialise the partial function p_1 to match f at Q_S (Step 1 in A_{MSW} and Step 1 in A_{GRY}). This, in turn, guaranteed consistency of hybrid oracles with Q_S throughout A_{MSW} 's and A_{GRY} 's execution. However, for private-coin setups, r_S is unavailable and we cannot proceed as above. We can also not afford to ignore Q_S as it may result in hybrid oracles that are *inconsistent* on Q_S : for example, if round j is such that A_{MSW} learns a query $m \in Q_S$, then a hybrid oracle h_s constructed as described in Equation (1) is likely to be inconsistent on Q_S (and thus pp). In this case, we won't be able to invoke perfect uniqueness (and correctness) of Π .

Two approaches to partially deal with this issue are presented in [MSW20]. The first approach works when the setup additionally is *efficient*. The second approach can handle expensive setups *provided* Π is also *tight*. We briefly discuss these two approaches and explain why they don't seem to readily extend to standard (i.e., non-tight) VDFs with *expensive* setup. In fact, [MSW20] leaves the impossibility of non-tight VDFs with expensive private-coin setup as an unresolved question; see Section 4.1 of [MSW20].

Exploiting setup efficiency. Recall from A_{MSW} 's description that we set $t_{on} = 6q_v$ to ensure that a random round i is likely to be one in which A_{MSW} does not learn a query from Q_V , and thus, h_s is consistent with f on Q_V . To ensure consistency of h_s *also* with f on Q_S in round i , we require A_{MSW} to not learn any query from $Q_V \cup Q_S$ in round i . By the same counting argument we used for A_{MSW} , there can be at most q_s rounds in which queries from Q_S can be learnt. Taken together, it suffices to therefore, say, set $t_{on} = 6(q_s + q_v)$. Since we assume efficient setup, $q_s \in \text{poly}(\log(T), \lambda)$ and thus the online phase runs for $6(q_v + q_s) \in \text{poly}(\log(T), \lambda)$ time, which is still within a valid sequentiality breaker's budget. However, for an *expensive* setup, where $q_s \in \text{poly}(T, \lambda)$, the sequentiality breaker is clearly invalid, i.e., runs in time greater than T .

Exploiting tightness. Recall that in a tight VDF, sequentiality holds even against adversaries whose online time complexity is close to the evaluation time T , say $T - \sqrt{T}$ for concreteness. In the second approach, [MSW20] exploits tightness to rule out ROM-based tight VDFs with even expensive private-coin setup.⁸ Keeping the goals of Section 2.4 in mind, we present their adversary in two steps. First we present an online adversary A'_{MSW} that breaks tight VDFs with *efficient* private-coin setup. Then we describe a preprocessing algorithm P_{MSW} , which lifts A'_{MSW} to break tight VDFs with *expensive* private-coin setup. Towards that end, we discuss how tightness is essential to the overall strategy, and why it does not seem to easily extend to standard (i.e., non-tight) VDFs.

The attack relies on the insight that `Verify` in a tight VDF Π is “insensitive” to small changes to `Eval`. Thus, to break uniqueness, A'_{MSW} can simply output $(y', \pi') \leftarrow \text{Eval}^f(\text{pp}, x)$, while *locally simulating* a small fraction of rounds of queries to f . An informal description follows.

Uniqueness Breaker A'_{MSW}

1. Sample a random set $\mathcal{S} \subseteq \{1, \dots, T\}$ of size $\sqrt{T}$
2. On input (pp, x) , run `Eval` on input (pp, x) with its i -th round oracle queries $(m_1, \dots, m_\ell)$ replied to as follows:

⁸ To be precise, [MSW20] rules out ROM-based tight PoSWs with expensive private-coin setup. Since a PoSW is simply a VDF *without* the uniqueness guarantee, their result also rules out tight VDFs with expensive private-coin setup. What we present is essentially their two-stage adversarial strategy for breaking sequentiality of tight PoSWs, suitably adapted for breaking uniqueness of tight VDFs (borrowing ideas from [DGMV20]).

- If $i \in \mathcal{S}$ then return $(r_1, \dots, r_\ell)$, where $r_1, \dots, r_\ell \leftarrow \{0, 1\}^\lambda$
 - Else return $(f(m_1), \dots, f(m_\ell))$.
3. Output (y', π') , which is **Eval**'s output in Step 2.

Let $(y, \pi) \leftarrow \text{Eval}^f(\text{pp}, x)$. Since A'_{MSW} runs in time $T - \sqrt{T}$, $y' \neq y$ holds with high probability due to Π 's sequentiality. Furthermore, as we explain next, $\text{Verify}^f(\text{pp}, x, y', \pi') = 1$ holds with high probability, resulting in A'_{MSW} breaking Π 's uniqueness. By “locally simulated” oracle, we refer to the oracle f' that results from local simulation of **Eval** queries (Step 2). As long as none of the queries in $Q_S \cup Q_V$ end up simulated, if **Verify** accepts (y', π') relative to f' then **Verify** accepts (y', π') relative to f too. It remains to bound the probability that a query in $Q_S \cup Q_V$ is simulated. This can be shown using a union bound to be at most $(q_s + q_v)/\sqrt{T}$ and thus **Verify** accepts (y', π') with probability at least

$$1 - \frac{(q_s + q_v)}{\sqrt{T}}. \quad (5)$$

Since $q_s + q_v \ll \sqrt{T}$, the value of Equation (5) is close to 1 for VDFs with efficient setup.

For expensive setups, the bound obtained in Equation (5) is meaningless and to resolve this issue, [MSW20] resorts to *preprocessing*. Deem a setup query “heavy” if it is likely to be queried again during evaluation. The goal of preprocessing is to learn the heavy setup queries, and [MSW20] proposes repeatedly running **Eval** on random inputs, say for $t_{\text{pre}} := 6q_s$ times, to collect such heavy queries. Their preprocessing algorithm is described below (suitably adapted for VDFs).

Preprocessing Algorithm P_{MSW}

1. *Initialise.* Set partial function p_{pre} to be empty.
2. *Learn.* On input pp , for each $j \in \{1, \dots, t_{\text{pre}}\}$:
 - (a) Sample random VDF input x_j and run $(y_j, \pi_j) \leftarrow \text{Eval}^f(\text{pp}, x_j)$.
 - (b) Let $Q^* := \{m_1, \dots, m_\ell\}$ denote the new queries made by **Eval** to f during this computation, where “new” is now with respect to p_{pre} .
 - (c) For each $m \in Q^*$, set $p_{\text{pre}}(m) := f(m)$.
3. Output p_{pre} .

In [MSW20], it is shown that most heavy setup queries will likely have been collected in p_{pre} , and it thus suffices to carry out the online phase as in A'_{MSW} , except with all queries in p_{pre} responded to honestly. They then show that the online phase, which involves a *single* invocation of **Eval**, is unlikely to involve a new setup query. While P_{MSW} requires $q_s q_e$ queries, the online phase has the same time complexity as A'_{MSW} , i.e., $T - \sqrt{T}$. It is thus possible to argue like before that if sequentiality holds then (y', π') breaks Π 's uniqueness.

Note however the idea underlying A'_{MSW} , i.e., locally simulate a fraction of **Eval** queries, limits it to tight VDFs: if the adversary is allowed only, say, $T/2$ time, then around $T/2$ rounds need to be locally simulated. This would once again render the union bound meaningless.

2.4 Handling *Expensive* Private-Coin Setups Using Preprocessing

We overcome the barriers explained in Section 2.3 using preprocessing. In particular, we use preprocessing to extend the [GRY25] result to *expensive private-coin* setups without having to rely on tightness. Our preprocessing algorithm, denoted P , is conceptually different from P_{MSW} and its design and analysis are our main technical contribution. In the remainder of this section, we explain why P_{MSW} falls short of serving our purpose, and how P resolves these shortcomings.

Recall from the analysis of A_{MSW} and A_{GRY} that we want to ensure A_{MSW} *does not* learn any query from $Q_V \cup Q_S$ in most online rounds. This allowed us to construct the consistent hybrid oracles, which aided breaking sequentiality or uniqueness. The goal of preprocessing is to generate a partial function p_{pre} such that the aforementioned guarantee is still likely to hold when A_{MSW} (and thus A_{GRY}) is executed in the online learning stage with the first-round partial function p_1 initialised to p_{pre} . That is, most of the t_{on} online rounds must be such that A_{MSW} (and thus A_{GRY}) does not learn any query from $Q_V \cup Q_S$.

To this end, let us first analyse the straw man proposal where we replace the preprocessing step in A_{MSW} (Step 1) and A_{GRY} (Step 1) with P_{MSW} from Section 2.3. The intention, as was for A'_{MSW} , is to learn the heavy setup queries. Recall however that A'_{MSW} involved a *single* call to $Eval$, and it sufficed to avoid queries from Q_S in this *particular* call. However, the online learning phase of A_{MSW} can involve t_{on} invocations of $Eval$, relative to different simulated (but not independent) random oracles $g_1, \dots, g_{t_{on}}$, and we need A_{MSW} to not learn queries from Q_S for most of these rounds. When $t_{pre} = 6q_s$, we are unable to rule out A_{MSW} making new setup queries in many rounds – in fact we believe this does not generally hold. P_{MSW} could potentially compensate for the $t_{on} = 6q_v$ invocations of $Eval$ by running its own learning phase (Step 2) for longer, say $t_{pre} = 6q_s q_v$. However, we were unable to analyse this strategy, or ascertain whether or not the guarantee even holds.

Our way around this issue is to use a different strategy for preprocessing, one which we *are actually able to analyse*. Our strategy implemented by P relies on the following crucial observation: it is not necessary to learn the heavy queries in Q_S that $Eval$ is likely to make, but only those heavy queries *that A_{MSW} (respectively, A_{GRY}) is likely to make in the online learning stage*. This is sufficient to ensure that most rounds in the online learning phase are where A_{MSW} (respectively A_{GRY}) does not learn queries from Q_S . Thus P repeatedly runs A_{MSW} instead of $Eval$ with the intention of learning the heavy queries that A_{MSW} itself is likely to make. An informal description of P follows – a formal description can be found in Algorithm 4. For our analysis, we set $t_{pre} = 6q_s$ and $t_{on} = 6q_v$.

Preprocessing Algorithm P

1. *Initialise.* Set partial function p_{pre} to be empty. Sample $i \leftarrow \{1, \dots, t_{pre}\}$.
2. *Learn online.* For each $j \in \{1, \dots, i\}$:

- (a) Sample random VDF input x_{pre} , and run online learning phase of A_{MSW} (Step 2) with $p_1 := p_{pre}$, parameter t_{on} and challenge x_{pre} to obtain y_{pre} .
 - (b) Let Q^* denote the new queries made by A_{MSW} to f during this computation, where “new” is again with respect to p_{pre} .
 - (c) For each $m \in Q^*$, set $p_{pre}(m) := f(m)$.
3. Output p_{pre} .

Note that, since A_{MSW} makes at most $6q_e q_v$ queries, the number of queries made by P is at most $36q_s q_e q_v$. Now when we run A_{MSW} in the online learning stage with $p_1 := p_{pre}$ generated by P , it is possible to show that in most rounds A_{MSW} *does not* learn any query from $Q_V \cup Q_S$. The formal analysis largely follows the “two-world” argument we used for A_{GRY} , but extra attention needs to be paid due to preprocessing. We show that the probability of A_{MSW} breaking sequentiality in the original world is roughly the same as the probability of A_{GRY} breaking uniqueness in the flipped world. We state and prove this in Lemma 3. We refer to Section 5 for the details.

3 Preliminaries

3.1 Notation

Sets. For $n \in \mathbb{N}$, let $[n] := \{1, \dots, n\}$, and let $\lambda \in \mathbb{N}$ be the security parameter. For a set X , we write $x \leftarrow X$ to denote that x is sampled from X uniformly at random.

Algorithms. For a probabilistic algorithm A , we write $y \leftarrow A(x)$, or sometimes also $A(x) \rightarrow y$, to denote that A run on input x with uniformly random coins outputs y . If we need to make the random coins explicit, we write $A(x; r)$ to denote that algorithm A is run on input x with random coins r .

Functions. We define the set of *partial functions* $\mathcal{P}_\lambda := \{p: \{0, 1\}^* \dashrightarrow \{0, 1\}^\lambda : \text{Dom}(p) \subseteq \{0, 1\}^*, \text{ i.e., the set of functions } p \text{ which are only defined on a subset } \text{Dom}(p) \subseteq \{0, 1\}^*. \text{ For a function set } \mathcal{F}_\lambda = \{f: \{0, 1\}^* \rightarrow \{0, 1\}^\lambda\} \text{ and a partial function } p \in \mathcal{P}_\lambda, \text{ we let } \mathcal{F}_\lambda|_p \text{ denote the subset of } \mathcal{F}_\lambda \text{ that agrees with } p, \text{ i.e., } \mathcal{F}_\lambda|_p = \{f \in \mathcal{F}_\lambda : f(x) = p(x) \forall x \in \text{Dom}(p)\}.$

We denote the set of polynomial functions $f: \mathbb{N}^n \rightarrow \mathbb{N}$ ($n \in \mathbb{N}$) by poly . A function $f: \mathbb{N} \rightarrow \mathbb{N}$ is negligible if its absolute value is smaller than the inverse of any polynomial (i.e., if $\forall c \exists k_0 \forall \lambda \geq k_0: |f(\lambda)| < 1/\lambda^c$). We denote the class of negligible functions by negl , and hence, we write $f \in \text{negl}$ or $f(\lambda) \in \text{negl}(\lambda)$ if f is a negligible function.

3.2 The Parallel Random Oracle Model (PROM)

The random oracle is defined to be the uniform distribution over the set $\mathcal{F}_\lambda$ of all functions $f: \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$. In this work, random oracles will be simulated lazily, i.e., whenever a new query x is made to f , $f(x)$ is set to a uniformly

random string $y \leftarrow \{0, 1\}^\lambda$. We write $f \leftarrow \mathcal{F}_\lambda$ for this process. In the proofs later we want to associate oracles with partially defined functions, i.e. they initially correspond to a completely undefined partial function and with every query the associated partial function gets updated.

An f -oracle-aided algorithm A^f , on input an initial state/input, runs in rounds: in the i -th round it (a) makes a tuple of parallel queries Q_i to f and receives $f(Q_i) := (f(x))_{x \in Q_i}$, and (b) runs an arbitrary computation on its current state and $f(Q_i)$, and passes the result as a state to the next round. For a k -round algorithm A^f we refer to $q := \sum_{i=1}^k |Q_i|$ as the (total) number of queries. If A^f is a deterministic algorithm, then $Q(A^f)$ denotes the set of queries A makes to f . We define the *time complexity* of A^f as the number of rounds it takes A^f to terminate.

3.3 Verifiable Delay Functions

A verifiable delay function (VDF) [BBBF18] is a function that requires many sequential steps to evaluate, yet the output (together with a proof) can be publicly verified much more efficiently. We define VDFs with respect to any oracle distribution $\mathcal{O}_\lambda$. VDFs in the ROM are obtained as a special case when $\mathcal{O}_\lambda := \mathcal{F}_\lambda$ where $\mathcal{F}_\lambda$ is defined in Section 3.2.

Definition 1 (Verifiable Delay Functions). *An oracle-aided Verifiable Delay Function (VDF) is a triple of oracle-aided (potentially randomized) uniform algorithms $\Pi := (\text{Setup}, \text{Eval}, \text{Verify})$, such that for any $\lambda \in \mathbb{N}$ and oracle distribution $\mathcal{O}_\lambda$ the following hold:*

- $\text{Setup}(\lambda, T) \rightarrow \text{pp}$: *On input a security parameter λ and time parameter T , it outputs the public parameters pp . The public parameters determine an efficiently samplable finite set of inputs X , a set of outputs Y , and a set of proofs P . We say that Π has a public-coin setup if pp also contains the random coins used by Setup . Otherwise, we say that Π has a private-coin setup.*
- $\text{Eval}(\text{pp}, x) \rightarrow (y, \pi)$: *On input pp and a value x , it outputs $y \in Y$ and a (possibly empty) proof $\pi \in P$. The first output y is determined only by x and pp , i.e., y is independent of the randomness of Eval .*
- $\text{Verify}(\text{pp}, x, y, \pi) \in \{0, 1\}$: *This is a deterministic algorithm that on input pp , values x, y , and a proof π , outputs a bit indicating whether it accepts or rejects.*

We require Π to satisfy efficiency, completeness, sequentiality and uniqueness as defined below.

- **Efficiency.** *Π is efficient if the following hold for every $O \in \mathcal{O}_\lambda$:*
 - Setup^O makes at most $q_s \in \text{poly}(T, \lambda)$ queries to O ,
 - Eval^O makes at most $q_e \in \text{poly}(T, \lambda)$ queries to O in at most T rounds.
 - Verify^O makes at most $q_v \in \text{poly}(\log(T), \lambda)$ queries to O .

- **α -Completeness.** Π is complete with error α if for any $\lambda, T \in \mathbb{N}$, every $O \in \mathcal{O}_\lambda$, any $\text{pp} \leftarrow \text{Setup}^O(\lambda, T)$, and any $x \in X$

$$\Pr \left[\text{Verify}^O(\text{pp}, x, y, \pi) = 1 : (y, \pi) \leftarrow \text{Eval}^O(\text{pp}, x) \right] \geq 1 - \alpha(\lambda).$$

We say Π satisfies perfect completeness if $\alpha = 0$.

- **Sequentiality.** Π satisfies sequentiality with $\epsilon \in [0, 1)$ gap if for every $\lambda, T \in \mathbb{N}$ and every two-stage oracle-aided adversary $\mathbf{A} = (\mathbf{A}_0, \mathbf{A}_1)$ such that $\mathbf{A}$ makes in total $\text{poly}(T, \lambda)$ oracle queries and $\mathbf{A}_1$ makes its queries in T^ϵ rounds, it holds that

$$\Pr \left[\begin{array}{l} y' = y : \\ \begin{array}{l} O \leftarrow \mathcal{O}_\lambda \\ \text{pp} \leftarrow \text{Setup}^O(\lambda, T) \\ \text{st} \leftarrow \mathbf{A}_0^O(\text{pp}) \\ x \leftarrow X \\ y' \leftarrow \mathbf{A}_1^O(\text{st}, x) \\ (y, \pi) \leftarrow \text{Eval}^O(\text{pp}, x) \end{array} \end{array} \right] \in \text{negl}(\lambda).$$

- **(Non-adaptive) Computational Uniqueness.** Π satisfies computational uniqueness if for any $\lambda, T \in \mathbb{N}$ and oracle-aided adversary $\mathbf{A}$ that makes $q \in \text{poly}(T, \lambda)$ oracle queries, it holds that

$$\Pr \left[\begin{array}{l} y' \neq y \wedge \text{Verify}^O(\text{pp}, x, y', \pi') = 1 : \\ \begin{array}{l} O \leftarrow \mathcal{O}_\lambda \\ \text{pp} \leftarrow \text{Setup}^O(\lambda, T) \\ x \leftarrow X \\ (y', \pi') \leftarrow \mathbf{A}^O(\text{pp}, x) \\ (y, \pi) \leftarrow \text{Eval}^O(\text{pp}, x) \end{array} \end{array} \right] \in \text{negl}(\lambda). \quad (6)$$

We say that Π satisfies perfect uniqueness if q is unbounded and Equation (6) holds with probability 0.

Remark 1. Some VDF definitions [BBBF18, Pie19, Wes19] use the word soundness to describe uniqueness. The two terms describe the same thing and we use uniqueness following [MSW20, GRY25]. We note that previous works [BBBF18, MSW20, DGMV20] defined uniqueness adaptively, i.e., instead of sampling x uniformly at random and giving x to the adversary $\mathbf{A}$ as input, they allow $\mathbf{A}$ to choose x itself. This adaptive uniqueness is stronger than our non-adaptive uniqueness, but since our focus in this work is on impossibility results, working with non-adaptive uniqueness implies stronger results (analogous to [GRY25]).

4 Warmup: Impossibility for Public-Coin Setup

In this section, we set up the stage for our main results which we present in Section 5. To do so, we first unify the language [MSW20] and [GRY25] use to describe their adversaries. To accommodate this unification, we provide a

Algorithm 1 The preprocessing algorithm P_{pub} for public-coin setup.

Input Public parameters $\mathbf{pp}$

Output A partial function $p_{pre} : \{0, 1\}^* \dashrightarrow \{0, 1\}^\lambda$

- 1: Initialize p_{pre} as the empty partial function with $\text{Dom}(p_{pre}) = \emptyset$
 - 2: Run $\text{Setup}^f(\lambda, T; r_S)$ with the random coins r_S contained in $\mathbf{pp}$ (cf. Definition 1)
 - 3: Set $p_{pre}(m) = f(m)$ for each query $m \in Q(\text{Setup}^f(\lambda, T; r_S))$
 - 4: **Return** p_{pre}
-

different analysis to the results of [GRY25]. We crucially rely on this new analysis in Section 5 to extend the results of [GRY25] to VDFs with private-coin and expensive setups.

In Section 4.1 we present the simplest case of perfectly unique VDFs with public-coin setup, as considered by [MSW20], and in Section 4.2, we consider the case of computationally unique VDFs with public-coin setup, as handled by [GRY25].

Let $\Pi := (\text{Setup}, \text{Eval}, \text{Verify})$ be a VDF in the ROM. At the heart of the results of both Section 4.1 and Section 4.2 is the sequentiality breaker (P_{pub}, A_s) where P_{pub} and A_s are formally depicted in Algorithms 1 and 2, respectively. This breaker is based on the sequentiality breaker of [MSW20], with the difference that in A_s we select the output from a uniformly random iteration instead of using majority-based output selection. This turns out to simplify the analysis.

The online phase of the adversary A_s^f with oracle access f , takes as input public parameters $\mathbf{pp}$, $x \in X$, and a partial function p_{pre} computed by the preprocessing algorithm P^f . The preprocessing algorithm for the public-coin setup case we consider in this section is simple: on input $\mathbf{pp}$, the algorithm P_{pub}^f simply runs the setup algorithm with the public random coins contained in $\mathbf{pp}$, and stores the queries made to f during this process in the partial function p_{pre} .

For our results regarding VDFs in the private-coin setup in Section 5, A_s is the same but we employ a more sophisticated preprocessing phase.

In Figure 1 we formally describe an execution of the sequentiality game with the sequentiality breaker (P, A_s) , where P is the preprocessing algorithm.

4.1 Perfectly Unique VDFs

In this section, we consider the case of perfectly unique VDFs Π with public-coin setup. For simplicity, we first assume that Π is also perfectly correct.

Preprocessing as per Algorithm 1 ensures that for all queries m made in $\text{Setup}^f(\lambda, T; r_S)$ we have $g_j(m) = f(m)$ for all $j \in [i]$. The argument for the sequentiality breaker of both [GRY25] and [MSW20], depicted in Algorithm 2, now works as follows.

⁹ The algorithm P_{pub} used in this section is actually deterministic, so it would ignore random coins r_P . However, in Section 5 we will use a randomized preprocessing algorithm P_{pub} .

Algorithm 2 The sequentiality breaker A_s^f , essentially [MSW20, Algorithm 1].

Input: Public parameters $\mathbf{pp}$, an element $x \in X$, and a partial function p_{pre}

Hard-Coded: Running time parameter t_{on}

Output: An element $y \in Y$

- 1: Set $p_1 := p_{pre}$
 - 2: Sample $i \leftarrow \{1, \dots, t_{on}\}$ uniformly at random
 - 3: **for** $j = 1, \dots, i$ **do**
 - 4: Lazily sample $g_j \leftarrow \mathcal{F}_\lambda|_{p_j}$
 - 5: Sample random coins r_j and run $(y'_j, \pi'_j) \leftarrow \text{Eval}^{g_j}(\mathbf{pp}, x; r_j)$
 - 6: Initialize p_{j+1} as the partial function with $p_{j+1}(m) = p_j(m)$ for $m \in \text{Dom}(p_j)$
 - 7: Set $p_{j+1}(m) = f(m)$ in parallel for every $m \in Q(\text{Eval}^{g_j}(\mathbf{pp}, x; r_j)) \setminus \text{Dom}(p_j)$
 - 8: **end for**
 - 9: **Return** y'_i
-

1. The challenger samples a random function $f \leftarrow \mathcal{F}_\lambda$.
2. The challenger samples random coins r_S and runs $\mathbf{pp} := \text{Setup}^f(\lambda, T; r_S)$.
3. The adversary samples random coins⁹ r_P and runs $p_{pre} := \mathbf{P}^f(\mathbf{pp}; r_P)$.
4. The challenger samples an input $x \leftarrow X$ uniformly at random.
5. The adversary runs $y'_i \leftarrow A_s^f(\mathbf{pp}, x, p_{pre})$.
6. The challenger samples random coins r_E and runs $(y, \pi) := \text{Eval}^f(\mathbf{pp}, x; r_E)$.
7. The adversary succeeds if and only if $y'_i = y$.

Fig. 1. The execution of the sequentiality game of Definition 1 with adversary (P, A_s) .

Consider the case that in the i -th iteration of Algorithm 2, i.e., the last iteration, no new query is made to g_i that is also made in $Q(\text{Verify}^f(\mathbf{pp}, x, y, \pi))$, i.e., we have

$$(Q(\text{Eval}^{g_i}(\mathbf{pp}, x; r_i)) \setminus \text{Dom}(p_i)) \cap Q(\text{Verify}^f(\mathbf{pp}, x, y, \pi)) = \emptyset. \quad (7)$$

We will sketch the proof of how A_s^f breaks sequentiality whenever Equation (7) holds. To do so, we construct a hybrid oracle h_s as

$$h_s(m) = \begin{cases} g_i(m) & \text{if } m \in Q(\text{Eval}^{g_i}(\mathbf{pp}, x; r_i)) \setminus \text{Dom}(p_i) \\ f(m) & \text{otherwise.} \end{cases} \quad (8)$$

By construction and perfect completeness, we now have

$$(y'_i, \pi'_i) = \text{Eval}^{h_s}(\mathbf{pp}, x; r_i) \quad \text{and} \quad \text{Verify}^{h_s}(\mathbf{pp}, x, y, \pi) = 1. \quad (9)$$

Hence, perfect uniqueness implies $y'_i = y$. Thereby, A_s^f breaks sequentiality whenever Equation (7) holds.

It remains to show that Equation (7) holds often. Recall that the oracles g_j get updated in each iteration and every query is made to f at most once. Furthermore, the number i of iterations of the for-loop is sampled uniformly at random from $[t_{on}]$. Since there are q_v queries made in $Q(\text{Verify}^f(\text{pp}, x, y, \pi))$, there can be at most q_v iterations where a query in $Q(\text{Verify}^f(\text{pp}, x, y, \pi))$ is made to f , hence, we obtain that i hits an iteration where Equation (7) is *not* satisfied with probability at most q_v/t_{on} , which results in a success probability for A_s^f of at least $1 - q_v/t_{on}$.

In the case that Π satisfies only α -completeness, Equation (9) only holds with probability $1 - \alpha$, which causes a loss of α in the adversary's advantage. We obtain the following result, which is, given that the exact construction of the sequentiality breaker differs slightly, essentially [MSW20, Theorem 3.1 and Section 4].

Theorem 2 (Corollary of [MSW20, Theorem 3.1 and Section 4]). *Let Π be a VDF in the ROM with public-coin setup satisfying perfect uniqueness and α -completeness, having running time parameters q_s , q_e and q_v , as in Definition 1. Let P_{pub} and A_s be the algorithms constructed in Algorithms 1 and 2, where the parameter t_{on} can be chosen as any positive natural number.*

Now, independently of the choice of $f \in \mathcal{F}_\lambda$, the pair (P_{pub}, A_s) wins the sequentiality game with probability at least $1 - q_v/t_{on} - \alpha$. Here, P_{pub} makes at most q_s queries to f , and A_s makes at most $q_e \cdot t_{on}$ queries to f , in at most t_{on} rounds.

4.2 Computationally Unique VDFs

Next we consider the case of computationally unique VDFs Π with public-coin setup. For simplicity, we first assume that Π satisfies perfect completeness. The argument of breaking sequentiality for perfectly unique VDFs fails when considering VDFs that are only *computationally* unique. This is so as it might happen that $y'_i \neq y$ while Equation (7) holds. However, the authors of [GRY25] observe that it is possible to break uniqueness with the same probability as this bad event is happening, i.e., they construct an adversary that breaks uniqueness with the same probability as the event

$$(Q(\text{Eval}^{g_i}(\text{pp}, x; r_i)) \setminus \text{Dom}(p_i)) \cap Q(\text{Verify}^f(\text{pp}, x, y, \pi)) = \emptyset \wedge y'_i \neq y \quad (10)$$

occurs for A_s^f . As $\text{Verify}^f(\text{pp}, x, y, \pi)$ makes only q_v queries, the first line of Equation (10) holds with probability at least $1 - q_v/t_{on}$ (as seen in Section 4.1), hence, the probability that Equation (10) holds is, up to an additive error of at most q_v/t_{on} , equal to the probability that the sequentiality breaker fails.

The uniqueness breaker of [GRY25] is (P_{pub}, A_u) where P_{pub} and A_u are depicted in Algorithms 1 and 3, respectively. Syntactically, the uniqueness breaker doesn't have a preprocessing phase, hence, the tuple (P_{pub}, A_u) is a (two-stage)

Algorithm 3 The uniqueness breaker A_u^f , essentially [GRY25, Algorithm 5].

Input: Public parameters $\mathbf{pp}$, an element $x \in X$, and a partial function p_{pre}

Hard-Coded: Running time parameter t_{on}

Output: An output-proof pair $(y, \pi) \in Y \times P$

- 1: Set $p_1 := p_{pre}$
 - 2: Sample $i \leftarrow \{1, \dots, t_{on}\}$.
 - 3: **for** $j = 1, \dots, i - 1$ **do**
 - 4: Lazily sample $g_j \leftarrow \mathcal{F}_\lambda|_{p_j}$
 - 5: Sample random coins r_j and run $(y'_j, \pi'_j) := \text{Eval}^{g_j}(\mathbf{pp}, x; r_j)$
 - 6: Initialize p_{j+1} as the partial function with $p_{j+1}(m) = p_j(m)$ for $m \in \text{Dom}(p_j)$
 - 7: Set $p_{j+1}(m) = f(m)$ in parallel for every $m \in Q(\text{Eval}^{g_j}(\mathbf{pp}, x; r_j)) \setminus \text{Dom}(p_j)$
 - 8: **end for**
 - 9: Sample random coins r_i and set $(y, \tilde{\pi}) := \text{Eval}^f(\mathbf{pp}, x; r_i)$
 - 10: Define a function h_u as follows. For $m \in (Q(\text{Eval}^f(\mathbf{pp}, x; r_i)) \setminus \text{Dom}(p_i))$, sample the value of $h_u(m)$ uniformly at random; otherwise, set $h_u(m) = f(m)$
 - 11: Sample random coins r_E and run $(y''_i, \pi''_i) := \text{Eval}^{h_u}(\mathbf{pp}, x; r_E)$
 - 12: **Return** (y''_i, π''_i)
-

uniqueness adversary. The first stage P_{pub} is the same as the sequentiality breaker preprocessing. The second stage A_s executes the exact same first $i - 1$ iterations of the sequentiality breaker of Algorithm 2, but then, in the last iteration, interchanges the roles of f and g_i , and constructs a function h_u such that running the evaluation algorithm with oracle access to h_u breaks uniqueness, i.e., $(y''_i, \pi''_i) = \text{Eval}^{h_u}(\mathbf{pp}, x; r_E)$ using random coins r_E is used as the tuple to break uniqueness.

Before sketching the proof of breaking uniqueness, let's recall the execution of the uniqueness game with the uniqueness breaker (P, A_u) in Figure 2.

The first step towards showing that (P_{pub}, A_u) indeed breaks uniqueness is to observe that if Equation (10) holds for A_s^f , then we have, by perfect correctness,

$$\text{Verify}^{h_s}(\mathbf{pp}, x, y, \pi) = \text{Verify}^f(\mathbf{pp}, x, y, \pi) = 1 \quad \wedge \quad y'_i \neq y. \quad (11)$$

Here, h_s is the hybrid oracle from Equation (8), $(y, \pi) = \text{Eval}^f(\mathbf{pp}, x; r_E)$ and $(y'_i, \pi'_i) = \text{Eval}^{h_s}(\mathbf{pp}, x; r_i)$. In other words, in case of perfect correctness, we obtain a break of uniqueness of the VDF (namely, the tuple (y, π)) *although only for the wrong oracle h_s , not f* . This in particular means, that for any f we can find an oracle h_s such that when (P_{pub}^f, A_s^f) fails to break sequentiality, then the original evaluation algorithm Eval^f succeeds in breaking uniqueness for h_s .

However, this is not sufficient for our final statement, for which we need to show that for a *uniformly random* f we have a non-negligible probability that either the sequentiality or the uniqueness breaker succeeds. (Note that the oracle

1. The challenger samples a random function $f \leftarrow \mathcal{F}_\lambda$.
2. The challenger samples random coins r_S and runs $\text{pp} := \text{Setup}^f(\lambda, T; r_S)$.
3. The challenger samples an input $x \leftarrow X$ uniformly at random.
4. The adversary samples random coins r_P and runs $p_{pre} := \mathbf{P}^f(\text{pp}; r_P)$.
5. The adversary runs $(y_i'', \pi_i'') \leftarrow \mathbf{A}_u^f(\text{pp}, x, p_{pre})$.
6. The adversary succeeds if and only if $\text{Verify}^f(\text{pp}, x, y_i'', \pi_i'') = 1$ and $y_i'' \neq y$ where y is the correct output, i.e., y is computed as $(y, \pi) \leftarrow \text{Eval}^f(\text{pp}, x)$.

Fig. 2. The execution of the uniqueness game of Definition 1 with adversary $(\mathbf{P}, \mathbf{A}_u)$.

h_s is not uniform and independent of $\mathbf{A}_s$'s random coins, but was made up by $\mathbf{A}_s^f$ itself!) To prove our final statement, it suffices to prove that the pair (y_i'', π_i'') , output by $(\mathbf{P}_{pub}^f, \mathbf{A}_u^f)$, when viewed as a random variable, breaks uniqueness with respect to the oracle f with *the same probability* as (y, π) , produced by Eval^f , breaks uniqueness with respect to h_s as implicitly sampled by $\mathbf{A}_s^f$. We prove this formally in Section 5.1 and only give a high-level overview below.

The proof utilized an auxiliary oracle f' that we define in terms of the lazily sampled f and g_i . Before defining f' , we need to make certain aspect of the sequentiality breaker/game more precise. At the beginning of the i -th iteration, all of the values of f that are determined are stored in the partial function p_i . Now, for $\mathbf{A}_s$, we call $\text{Eval}^{g_i}(\text{pp}, x; r_i)$ where g_i is a function satisfying $g_i(m) = p_i(m) = f(m)$ for $m \in \text{Dom}(p_i)$. During this call, queries at new values $m_1, \dots, m_\ell \in \{0, 1\}^* \setminus \text{Dom}(p_i)$ might come up. Whenever such a query is made, we sample a new random value $\gamma_{i,j} \leftarrow \{0, 1\}^\lambda$ such that at the end of the i -th iteration we have that

$$g_i(m_j) = \gamma_{i,j} \quad \text{for each } j \in \{1, \dots, \ell\}.$$

Later in the i -th iteration of $\mathbf{A}_s$, the values of f are determined. We then have

$$f(m_j) = \phi_{i,j} \quad \text{for each } j \in \{1, \dots, \ell\},$$

where $\phi_{i,1}, \dots, \phi_{i,\ell} \in \{0, 1\}^\lambda$ are all sampled uniformly at random.

With these assignments for f and g_i , let's consider the *uniqueness* breaker/game, but with respect to a different oracle f' , rather than f , where f' is defined by a function $f' : \{0, 1\}^* \dashrightarrow \{0, 1\}^\lambda$ where

$$f'(m) := \begin{cases} f(m) & \text{if } m \in \text{Dom}(p_i) \\ \gamma_{i,j} & \text{if } m = m_j \end{cases}.$$

Note that in this case, we have

$$\{m_1, \dots, m_\ell\} = \mathcal{Q}(\text{Eval}^{f'}(\text{pp}, x; r_i)).$$

If then, as in the last few lines of Algorithm 3, the function h_u is sampled as

$$h_u(m) = \begin{cases} \phi_{i,j} & \text{if } m = m_j \\ f'(m) & \text{otherwise,} \end{cases}$$

then by inspection we have $(f, h_s) = (h_u, f')$. That is, what was f in the original case is now h_u , and what was h_s is now f' .

We now observe that if the pair (f, h_s) makes Equation (11) occur in the sequentiality game, then in the uniqueness game (and assuming all other random coins, in particular r_i, r_E , are the same in the two experiments), with respect to f' , we have:

$$\text{Verify}^{f'}(\text{pp}, x, y_i'', \pi_i'') = \text{Verify}^{f'}(\text{pp}, x, y'', \pi'') = 1 \quad \wedge \quad y_i'' = y \neq y_i' = y'', \quad (12)$$

where we denoted $(y'', \pi'') := \text{Eval}^{f'}(\text{pp}, x; r_i)$. Stated differently, Equation (12) shows that the pair (y_i'', π_i'') breaks uniqueness with respect to the oracle f' . It remains to show that Equation (12) holds. This follows from observing that

$$\begin{aligned} h_u = f &\Rightarrow (y_i'', \pi_i'') := \text{Eval}^{h_u}(\text{pp}, x; r_E) = \text{Eval}^f(\text{pp}, x; r_E) =: (y, \pi) \\ h_s = f' &\Rightarrow (y_i', \pi_i') := \text{Eval}^{h_s}(\text{pp}, x; r_i) = \text{Eval}^{f'}(\text{pp}, x; r_i) \end{aligned}$$

and that $y_i' = y''$, which holds since by definition of VDFs, the first output of $\text{Eval}^{f'}$ is deterministically computed, and hence does not depend on r_i resp. r_E .

The crucial point now is that it is equally likely that the pair (f, h_s) gets sampled in the sequentiality game, as it is that the pair (f', h_u) gets sampled in the uniqueness game: (f, h_s) occurs if the challenger samples values $\phi_{i,j}$ and A_s samples values $\gamma_{i,j}$, whereas (f', h_u) occurs if A_u samples values $\phi_{i,j}$ and the challenger samples values $\gamma_{i,j}$, where both $\gamma_{i,j}$ and $\phi_{i,j}$ are independently and uniformly distributed over $\{0, 1\}^\lambda$. In other words, the pairs of random variables (f, h_s) and (h_u, f') are equally distributed. Furthermore, if Equation (10) occurs for A_s for (f, h_s) then A_u breaks uniqueness for (h_u, f') . Hence, this implies that A_u^f breaks uniqueness with at least the same probability as Equation (10) happens in the sequentiality game. It is important to stress that A_u does not break uniqueness *in case* Equation (10) holds, but rather it breaks uniqueness with (at least) the *same probability* as that of Equation (10) holding. In other words, the probability of A_u winning the uniqueness game is, up to an additive factor q_v/t_{on} , the same as that of A_s failing in the sequentiality game.

If Π only satisfies α -completeness, Equation (11) only holds with probability $1 - \alpha$. This causes a loss of α in the success rate of the uniqueness breaker. We obtain the following result, which is essentially the statement of [GRY25, Theorem 5.1]. Their constructions of the adversaries differ slightly, resulting in slightly different bounds.

Theorem 3 (Essentially [GRY25, Theorem 5.1]). *Let Π be a VDF in the ROM with public-coin setup and α -completeness, having running time parameters q_s , q_e and q_v , as in Definition 1. Let P_{pub} , A_s and A_u be the algorithms constructed in Algorithms 1 to 3, where the parameter t_{on} can be chosen as any positive natural number.*

If (P_{pub}, A_s) breaks sequentiality of Π with probability ε_s ,¹⁰ then (P_{pub}, A_u) breaks uniqueness of Π with probability at least $1 - \varepsilon_s - q_v/t_{on} - \alpha$. In other words, Π either fails to satisfy sequentiality or computational uniqueness. Furthermore, the complexity of these algorithms is as follows.

- A_s runs in at most t_{on} rounds and makes at most q_e queries in each round.
- A_u makes at most $q_e \cdot (t_{on} + 1)$ queries in total.
- P_{pub} makes at most q_s queries.

5 Our Results: Impossibility for Private-Coin Setup

In this section, we extend the known impossibility result for computationally unique VDFs with public-coin setup to setup algorithms that are both private-coin and expensive. The general outline of the argument follows that of Section 4.2: we construct a sequentiality breaker and a uniqueness breaker such that either the sequentiality or the uniqueness breaker succeeds with a non-negligible probability.

However, unlike the result and analysis of Section 4, the setup algorithm is private-coin, i.e., neither the sequentiality nor the uniqueness breaker gets to see the random coins used by the setup algorithm to sample the VDF’s public parameters. The challenge of extending the impossibility result of VDFs with public-coin to private-coin setup is to learn, by some way or another, most of the queries made by the private-coin setup. We discuss this matter in more detail after we discuss the preprocessing algorithm and its effect on the success probability of our adversaries.

Formally, assume towards contradiction that $(\text{Setup}, \text{Eval}, \text{Verify})$ is a VDF in the ROM. We construct a sequentiality breaker (P_{priv}, A_s) and a uniqueness breaker¹¹ (P_{priv}, A_u) such that at least one of them succeeds with a non-negligible probability. The algorithms A_s , A_u , and P_{priv} are formally depicted in Algorithms 2, 3, and 4, respectively.

As alluded to before, the challenge of extending the impossibility of VDFs with public-coin setup from Section 4 to those with private-coin setup, regardless of whether the setup is expensive or not, comes from the need to not only consider the overlap of the set of new queries made of the sequentiality/uniqueness

¹⁰ Here, probabilities are always taken over the choice of $f \leftarrow \mathcal{F}_\lambda$, the randomness used in the generation of the public parameters pp , the random choice of the input x and all of the randomness used during the execution of the adversaries.

¹¹ Syntactically, the uniqueness breaker in the uniqueness game (cf. Definition 1) is a single-stage adversary with no assigned meaning to preprocessing. We view (P_{priv}, A_u) as the uniqueness adversary that first runs P_{priv} and then A_u (using output of P_{priv}).

breaking algorithms with the set of verification queries, but also to consider the overlap of such queries with the (unknown) set of setup queries. This in particular requires a more sophisticated preprocessing algorithm than the one considered in Section 4. We formally describe preprocessing in $\mathcal{P}_{priv}$, Algorithm 4, and discuss it later after Theorem 4.

Looking ahead, the success probability of our uniqueness breaker will depend on two events: the event that the sequentiality breaker fails and a specific event E that may or may not happen during the execution of the sequentiality breaker.

We define the event E as the conjunction of two events E_1 and E_2 , where E_1 considers the overlap of new queries made during the i -th iteration of the sequentiality breaker with the queries made by the *verification* algorithm and E_2 considers such overlap with the queries made by the *private-coin setup*. E is formally given in Definition 2.

Definition 2 (Event E). *Interpreting the variables in the execution of the sequentiality game in Figure 1 with the adversary $\mathbf{A} = (\mathcal{P}_{priv}, \mathbf{A}_s)$ (cf. Algorithms 2 and 4) as random variables, let $E := E_1 \wedge E_2$ where events E_1 and E_2 are defined as follows.*

– E_1 is the event that

$$(\mathcal{Q}(\text{Eval}^{g_i}(\text{pp}, x; r_i)) \setminus \text{Dom}(p_i)) \cap \mathcal{Q}(\text{Verify}^f(\text{pp}, x, y, \pi)) = \emptyset.$$

– E_2 is the event that

$$(\mathcal{Q}(\text{Eval}^{g_i}(\text{pp}, x; r_i)) \setminus \text{Dom}(p_i)) \cap \mathcal{Q}(\text{Setup}^f(\lambda, T; r_S)) = \emptyset.$$

Note that if the sequentiality breaker $(\mathcal{P}_{priv}, \mathbf{A}_s)$ fails, i.e., it outputs $y'_i \neq y$, but the event E happens, then $(\mathcal{P}_{priv}, \mathbf{A}_s)$ breaks uniqueness w.r.t. (the wrong) oracle h_s defined in Equation (8) in Section 4.¹² Using an argument about the distributions occurring in the executions of the sequentiality and uniqueness games with the adversaries $(\mathcal{P}_{priv}, \mathbf{A}_s)$ and $(\mathcal{P}_{priv}, \mathbf{A}_u)$, respectively, we can bound the success probability of $(\mathcal{P}_{priv}, \mathbf{A}_u)$ in terms of the probabilities that $y'_i = y$ (i.e. $(\mathcal{P}_{priv}, \mathbf{A}_s)$ wins) and event E occurs. We formalize this in the following theorem and refer to Section 5.1 for its proof.

Theorem 4. *Let $\Pi := (\text{Setup}, \text{Eval}, \text{Verify})$ be an α -complete VDF in the ROM, $\mathbf{A}_s, \mathbf{A}_u$ and $\mathcal{P}_{priv}$ be as in Algorithms 2, 3, and 4, respectively, and fix preprocessing and running time parameters t_{pre} and t_{on} , respectively. If ε_s is the probability that the tuple $(\mathcal{P}_{priv}, \mathbf{A}_s)$ breaks sequentiality of Π , then the tuple $(\mathcal{P}_{priv}, \mathbf{A}_u)$ breaks uniqueness of Π with probability $\Pr[E] - \varepsilon_s - \alpha$, where E is defined in Definition 2.*

We now show how to bound $\Pr[E]$, resulting in the impossibility result for VDFs in the ROM given as Corollary 1. As $E = E_1 \wedge E_2$, we have $\Pr[E] \geq 1 - \Pr[\overline{E_1}] - \Pr[\overline{E_2}]$. Now we can bound $\Pr[\overline{E_1}]$ by a simple counting argument (see the case of perfect uniqueness above), i.e. $\Pr[\overline{E_1}] \leq q_v/t_{on}$.

¹² Note that in the case of public-coin setup (as considered in Section 4), E_2 is always true. This is why we only considered event E_1 in that case.

Lemma 1 (Event E_1). *With event E_1 as in Definition 2, t_{on} as in Theorem 4 and q_v as in Definition 1, we have*

$$\Pr[\overline{E_1}] \leq q_v/t_{on} .$$

Proof. (of Lemma 1) This follows the same line of arguments as we have seen in Section 4.1. Recall that the oracles g_j simulated by A_s^f are consistent with the partial functions p_j , which get updated in each iteration. During this process, every query is made to f at most once. Furthermore, the number i of iterations is sampled uniformly at random from $[t_{on}]$. Since there are q_v queries made in $\text{Verify}^f(\text{pp}, x, y, \pi)$, there can be at most q_v of the t_{on} iterations in which such a query is made. Hence, we obtain that the new queries made in round i hit a query in $Q(\text{Verify}^f(\text{pp}, x, y, \pi))$ with probability at most q_v/t_{on} . $\square$

Now, how do we bound $\Pr[\overline{E_2}]$? Here the preprocessing algorithm P_{priv} depicted in Algorithm 4 comes into play. Informally, P_{priv} *simulates* a number of executions of A_s on random inputs. The purpose of this simulation is to find most queries made by $\text{Setup}^f(\lambda, T; r_S)$, with *secret* random coins r_S , that are relevant for the execution of our adversaries. The more simulated executions of A_s that P_{priv} makes, the higher the chance it finds all such relevant queries, and hence the lower the chance that E_2 holds.

To make this precise, we first define an event D , which implies E_2 and captures the event of P_{priv} finding all queries made by setup that are relevant to our adversaries.

Definition 3 (Event D). *Interpreting the variables in the execution of the sequentiality game in Figure 1 with the adversary $A = (P_{priv}, A_s)$ (cf. Algorithms 2 and 4) as random variables, let D be the event that during the i -th execution of A_s in Line 4 of Algorithm 4, no new query to f at a value $m \in Q(\text{Setup}^f(\lambda, T; r_S))$ is made.*

Note that D implies E_2 , as any new query made by $\text{Eval}^{g_i}(\text{pp}, x; r_i)$, as part of A_s , is later also made to f . We can bound $\Pr[\overline{D}]$ with an argument very similar to the one used to bound $\Pr[\overline{E_1}]$ in Lemma 1.

Lemma 2 (Event D). *With event D as in Definition 3, t_{pre}, t_{on} as in Theorem 4 and q_s as in Definition 1, we have*

$$\Pr[\overline{D}] \leq q_s/t_{pre} .$$

Proof. Consider an execution of $P_{priv}^f(\text{pp}, t_{pre}, t_{on})$ with i set to t_{pre} in Line 2. No query is made to f twice, as each query made to f is immediately stored in p_{pre} . Thereby, each $m \in Q(\text{Setup}^f(\lambda, T; r_S))$ can occur in at most one of the i iterations. In particular, in at most q_s of the t_{pre} iterations of Line 4 of Algorithm 4 a new query to f at a value $m \in Q(\text{Setup}^f(\lambda, T; r_S))$ is made. Thereby, since $i \leftarrow [t_{pre}]$ is sampled uniformly at random, the probability of D occurring is at most q_s/t_{pre} . The claim follows. $\square$

Algorithm 4 The preprocessing algorithm P_{priv}^f for private-coin setup.

Input Public parameters $\mathbf{pp}$

Hard-Coded: Running time parameters t_{pre}, t_{on}

Output A partial function $p_{pre}: \{0, 1\}^* \dashrightarrow \{0, 1\}^\lambda$

- 1: Initialize p_{pre} as the empty partial function with $\text{Dom}(p_{pre}) = \emptyset$
 - 2: Sample $i \leftarrow \{1, \dots, t_{pre}\}$
 - 3: **for** $j = 1, \dots, i$ **do**
 - 4: Run $A_s^f(\mathbf{pp}, x_{pre}, p_{pre})$ from Algorithm 2 for a uniformly random $x_{pre} \leftarrow X$
 - 5: Set $p_{pre}(m) = f(m)$ for each query $m \notin \text{Dom}(p_{pre})$ made by A_s^f to f
 - 6: **end for**
 - 7: **Return** p_{pre}
-

Corollary 1 (VDFs cannot exist in the ROM). *Let $\Pi, A_s, A_u, P_{priv}, t_{pre}, t_{on}$ be as in Theorem 4, and q_s, q_v as in Definition 1. If (P_{priv}, A_s) breaks sequentiality of Π with probability¹³ ε_s , then (P_{priv}, A_u) breaks uniqueness of Π with probability at least $1 - \varepsilon_s - q_v/t_{on} - q_s/t_{pre} - \alpha$. That is, Π either fails to satisfy sequentiality or computational uniqueness. Furthermore, complexity of these algorithms is as follows.*

- A_s runs in at most t_{on} rounds and makes at most q_e queries in each round.
- A_u makes at most $q_e(t_{on} + 1)$ queries in total.
- P_{priv} runs in at most $t_{pre} \cdot t_{on}$ rounds and makes at most q_e queries in each round.

For example, we can fix $t_{pre} = 6q_s$, $t_{on} = 6q_v$. (Recall that by Definition 1, $q_e \in \text{poly}(T, \lambda)$, $q_v \in \text{poly}(\log(T), \lambda)$ and setup is allowed to be expensive, i.e., $q_s \in \text{poly}(T, \lambda)$, otherwise it is efficient, i.e., $q_s \in \text{poly}(\log(T), \lambda)$.) It follows that at least one of the following holds:

- (P_{priv}, A_s) breaks sequentiality with probability at least $1/3 - \alpha/2$. Note that if setup is efficient, i.e., $q_s \in \text{poly}(\log(T), \lambda)$, then so is the preprocessing, i.e., P_{priv} runs in $\text{poly}(\log(T), \lambda)$ rounds with a total of $\text{poly}(T, \lambda)$ queries. Regardless of the (in)efficiency of setup, the online adversary is efficient, i.e., A_s runs in $\text{poly}(\log(T), \lambda)$ rounds with a total of $\text{poly}(T, \lambda)$ queries. $q_s \in \text{poly}(\log(T), \lambda)$, then so is $(t_{pre}) \cdot t_{on}$.
- (P_{priv}, A_u) breaks uniqueness with probability at least $1/3 - \alpha/2$. Note that in general, (P_{priv}, A_u) runs in a total of $\text{poly}(T, \lambda)$ rounds and queries.

Proof. (of Corollary 1) As $E = E_1 \wedge E_2$, we have $\Pr[E] \geq 1 - \Pr[\overline{E_1}] - \Pr[\overline{E_2}]$. Given that event D implies E_2 , Lemmas 1 and 2 imply $\Pr[E] \geq 1 - q_v/t_{on} - q_s/t_{pre}$. Theorem 4 implies that (P_{priv}, A_u) breaks uniqueness with probability

$$\Pr[E] - \varepsilon_s - \alpha \geq 1 - \varepsilon_s - q_v/t_{on} - q_s/t_{pre} - \alpha.$$

¹³ The probability is taken over the choice of $f \leftarrow \mathcal{F}$, the randomness used in the generation of the public parameters $\mathbf{pp}$, the random choice of the input x and all of the randomness used while running the adversaries.

Plugging in $t_{pre} = 6q_s$, $t_{on} = 6q_v$ leads to the final claim. $\square$

5.1 The Success Rate of the Computational Uniqueness Breaker

The aim of this section is to prove Theorem 4. More precisely, we prove that, denoting the success rate of the uniqueness breaker (P_{priv}, A_u) —as depicted in Algorithms 3 and 4—by ε_u , and the success rate of the sequentiality breaker (P_{priv}, A_s) —as depicted in Algorithms 3 and 4—by ε_s , the bound

$$\varepsilon_s + \varepsilon_u \geq 1 - \Pr[E] - \alpha$$

holds. Here, E is the event defined in Definition 2, and α is the completeness parameter. To this aim, analogous to event E for the sequentiality game (Definition 2), we introduce the event $E' = E'_1 \wedge E'_2$ for the uniqueness game. The events E'_1 and E'_2 are very similar to E_1 and E_2 from Definition 2, the parts where they differ from their respective counterparts are color-coded below.

Definition 4 (Event E'). *Interpreting the variables in the execution of the sequentiality game in Figure 2 with the adversary $A = (P_{priv}, A_u)$ (cf. Algorithms 3 and 4) as random variables, let $E' := E'_1 \wedge E'_2$ where events E'_1 and E'_2 are defined as follows.*

– E'_1 is the event that

$$\left(Q(\text{Eval}^f(\text{pp}, x; r_i)) \setminus \text{Dom}(p_i) \right) \cap Q(\text{Verify}^{h_u}(\text{pp}, x, y_i'', \pi_i'')) = \emptyset.$$

– E'_2 is the event that

$$\left(Q(\text{Eval}^f(\text{pp}, x; r_i)) \setminus \text{Dom}(p_i) \right) \cap Q(\text{Setup}^f(\lambda, T; r_S)) = \emptyset.$$

Lemma 3. *Let $\Pi := (\text{Setup}, \text{Eval}, \text{Verify})$ be an α -complete VDF in the ROM, A_u and P_{priv} be as in Algorithms 3 and 4, respectively, and fix preprocessing and running time parameters t_{pre} and t_{on} , respectively. Then the tuple (P_{priv}, A_u) breaks uniqueness of Π with probability $\Pr[(y_i'' \neq y) \wedge E'] - \alpha$, where E' is defined in Definition 4.*

Proof of Lemma 3. We first show that

$$\Pr \left[\text{Verify}^f(\text{pp}, x, y_i'', \pi_i'') = 1 \mid E' \right] \geq 1 - \alpha. \quad (13)$$

To this end, note that f and h_u only differ at values $m \in (Q(\text{Eval}^f(\text{pp}, x; r_i)) \setminus \text{Dom}(p_i))$. If E' occurs, then in particular E'_2 occurs, implying that for every $m \in Q(\text{Setup}^f(\lambda, T; r_S))$, we have $h_u(m) = f(m)$. Hence,

$$\text{pp} := \text{Setup}^f(\lambda, T; r_S) = \text{Setup}^{h_u}(\lambda, T; r_S).$$

Since $(y_i'', \pi_i'') := \text{Eval}^{h_u}(\text{pp}, x; r_E)$ and Π satisfies α -completeness, we find that

$$\Pr \left[\text{Verify}^{h_u}(\text{pp}, x, y_i'', \pi_i'') = 1 \mid E_2' \right] \geq 1 - \alpha.$$

Now, if $\text{Verify}^{h_u}(\text{pp}, x, y_i'', \pi_i'') = 1$ and E_1' occurs, then for all queries m made by $\text{Verify}^{h_u}(\text{pp}, x, y_i'', \pi_i'')$, we have $h_u(m) = f(m)$, and we therefore obtain $\text{Verify}^f(\text{pp}, x, y_i'', \pi_i'') = 1$. This implies Equation (13). If additionally $y \neq y_i''$ holds, then $(\text{P}_{\text{priv}}, \text{A}_u)$ breaks uniqueness of Π , and the claim follows. $\square$

For the proof of Theorem 4, it remains to prove the inequality

$$\Pr[(y_i'' \neq y) \wedge E'] \geq \Pr[E] - \varepsilon_s. \quad (14)$$

We do so by proving the equality

$$\Pr[(y_i'' \neq y) \wedge E'] = \Pr[(y_i' \neq y) \wedge E], \quad (15)$$

which gives

$$\begin{aligned} \Pr[(y_i'' \neq y) \wedge E'] &= \Pr[(y_i' \neq y) \wedge E] = \Pr[E] - \Pr[(y_i' = y) \wedge E] \\ &\geq \Pr[E] - \Pr[y_i' = y] = \Pr[E] - \varepsilon_s. \end{aligned}$$

We prove Equation (15) by carefully analyzing the randomness used during the execution of both the sequentiality game with $(\text{P}_{\text{priv}}, \text{A}_s)$ and the computational uniqueness game with $(\text{P}_{\text{priv}}, \text{A}_u)$. We analyze the random coins that determine whether event $y_i' = y$ and event E , resp. $y_i'' \neq y$ and E' occur.

To this end, we define the notion of *random tapes*. For us, a random tape is a triple $\mathbf{t} = (\mathbf{t}_1, \mathbf{t}_2, \mathbf{t}_3)$, where

1. $\mathbf{t}_1 \in \{1, \dots, t_{\text{on}}\}$ determines the value of i .
2. $\mathbf{t}_2 = (r_S, r_P, r_x, r_1, \dots, r_i, r_E)$ denotes the random coins sampled during an execution of the sequentiality game with the adversary $(\text{P}_{\text{priv}}, \text{A}_s)$ (see Figure 1), or the uniqueness game with the adversary $(\text{P}_{\text{priv}}, \text{A}_u)$ (see Figure 2). That means, r_S is used to run **Setup**, r_P to run P_{priv} , r_x to sample $x \leftarrow X$. Randomness r_j for $j \in [i]$ and r_E is used, depending on whether the sequentiality or the uniqueness game is considered, for the i runs of **Eval** by A_s and the run of **Eval** by the challenger, or for the $(i+1)$ respective **Eval** calls within A_u .
3. $\mathbf{t}_3$ determines the values of the random functions f and $(g_j)_{1 \leq j \leq i}$, both of which we interpret as lazily determined. We formally accomplish this by writing $\mathbf{t}_3 = (\Phi, \Gamma)$, where $\Phi = (\phi_S, \phi_P, \phi_1, \dots, \phi_i, \phi_E)$ and $\Gamma = (\gamma_1, \dots, \gamma_i)$, where furthermore $\phi_S := (\phi_{S,1}, \dots, \phi_{S,q_s}) \in \{0, 1\}^{\lambda \times q_s}$ denotes the tape used to sample the random values of f at the values queried by $\text{Setup}^f(\lambda, T; r_S)$. Here, when f is queried at a first value $m_1 \in \{0, 1\}^*$, we have $f(m_1) = \phi_{S,1}$, etc. Similarly, ϕ_P denotes the tape used to sample the random images of values queried during the execution of $\text{P}_{\text{priv}}^f(\text{pp}; r_P)$, and γ_j and ϕ_j denote, for $1 \leq j \leq i$, elements in $\{0, 1\}^{\lambda \times q_e}$, determining the random image values

of g_j and f on input values queried in round j , in each of the algorithms. Moreover, γ_i and ϕ_i denote elements in $\{0, 1\}^{\lambda \times q_e}$, determining the random image values of g_i and f on input values queried in the i -th round of A_s , and of h_u and f in the i -th round of A_u .¹⁴ Finally, $\phi_E \in \{0, 1\}^{\lambda \times q_e}$ denotes the image values of f for the sequentiality game, resp. for h_u for the uniqueness game, on new queries coming up in the respective executions of **Eval** with random coins r_E , which is done by the challenger in case of the sequentiality game, and by A_u in case of the uniqueness game.

We denote the set of all possible tapes by **Tapes**. This is a finite set, and it is naturally equipped with a (uniform) probability measure. Any tape $\mathbf{t} = (t_1, t_2, t_3) \in \mathbf{Tapes}$ provides enough random coins to run all of the sequentiality game, and enough random coins to completely run A_u in the uniqueness game and decide whether events $y'_i = y$ and E , or $y''_i = y$ and E' , occur, respectively. We write $\mathbf{t} \implies (y'_i \neq y) \wedge E$ if $(y'_i \neq y) \wedge E$ occurs when the sequentiality game uses the randomness specified in $\mathbf{t}$. Similarly, we write $\mathbf{t} \implies (y''_i \neq y) \wedge E'$ if $(y''_i \neq y) \wedge E'$ occurs when the computational uniqueness game uses the randomness specified in $\mathbf{t}$.

Given a tape $\mathbf{t} = (t_1, t_2, t_3)$, we denote by $\mathbf{t}'$ the tape (t'_1, t'_2, t'_3) where $t'_1 = t_1$, $t'_2 = t_2$ and t'_3 is constructed as follows. Recall that we write $t_3 = (\Phi, \Gamma)$ for $\Phi = (\phi_S, \phi_P, \phi_1, \dots, \phi_i, \phi_E)$ and $\Gamma = (\gamma_1, \dots, \gamma_i)$. Now, we define t'_3 as the tape obtained by swapping ϕ_i and γ_i , i.e., $t'_3 = (\Phi', \Gamma')$ where $\Phi' = (\phi_S, \phi_P, \phi_1, \dots, \phi_{i-1}, \gamma_i, \phi_E)$ and $\Gamma' = (\gamma, \dots, \gamma_{i-1}, \phi_i)$. Clearly, we have $\Pr[\{\mathbf{t}\}] = \Pr[\{\mathbf{t}'\}]$ for tape $\mathbf{t} \in \mathbf{Tapes}$.

Lemma 4. *For any $\mathbf{t} \in \mathbf{Tapes}$,*

$$(\mathbf{t} \implies (y'_i \neq y) \wedge E) \iff (\mathbf{t}' \implies (y''_i \neq y) \wedge E').$$

In particular, we have $\Pr[(y'_i \neq y) \wedge E] = \Pr[(y''_i \neq y) \wedge E']$.

Proof. We denote by $g_i^{\mathbf{t}}$ the function g_i as determined by $\mathbf{t}$. We use the same notation for other random variables, if the passage from $\mathbf{t}$ to $\mathbf{t}'$ makes a difference.

If we use tape $\mathbf{t}$ to run the sequentiality game, then the event E_2 reads $Q(\text{Eval}^{g_i^{\mathbf{t}}}(\text{pp}, x; r_i) \setminus \text{Dom}(p_i)) \cap Q(\text{Setup}^{f^{\mathbf{t}}}(\lambda, T; r_S)) = \emptyset$. This means that none of the $f^{\mathbf{t}}$ -images of the new queries made to $g_i^{\mathbf{t}}$ by $\text{Eval}^{g_i^{\mathbf{t}}}(\text{pp}, x; r_i)$ was set already in the setup phase, hence, in particular, at the beginning of the i -th iteration, $f^{\mathbf{t}}$ is not yet determined on those queries. Now, if we denote by $f^{\mathbf{t}'}$ the function f as determined when using $\mathbf{t}'$ to run the computational uniqueness game, hence

$$f^{\mathbf{t}'}(m) = g_i^{\mathbf{t}}(m) \text{ for } m \in Q(\text{Eval}^{g_i^{\mathbf{t}}}(\text{pp}, x; r_i)). \quad (16)$$

¹⁴ Note that the *set of queries* made in round i depends on γ_i in the sequentiality game, whereas it depends on ϕ_i in the soundness game. This will be crucial in the proof of Lemma 4.

This in particular implies $Q(\text{Eval}^{g_i^t}(\text{pp}, x; r_i)) \setminus \text{Dom}(p_i) = Q(\text{Eval}^{f^{t'}}(\text{pp}, x; r_i)) \setminus \text{Dom}(p_i)$, which immediately implies that t' makes E'_2 occur. The reverse direction can be shown exactly the same way, hence we obtain

$$(t \implies E_2) \iff (t' \implies E'_2).$$

Again we use t to run the sequentiality game and t' to run the computational uniqueness game, and now assume $t \implies ((y'_i \neq y) \wedge E)$. In particular, we have $t \implies E_2$, and by Equation 16 the same sequence of queries is made in $\text{Eval}^{g_i^t}(\text{pp}, x; r_i)$ and in $\text{Eval}^{f^{t'}}(\text{pp}, x; r_i)$. Hence, exactly the same random coins that are used to sample f^t in the sequentiality game are used to sample $h_u^{t'}$ in the computational uniqueness game, and vice versa. More concretely, we have $f^t = h_u^{t'}$ and $g_i^t(m) = f^{t'}(m)$ for $m \in Q(\text{Eval}^{g_i^t}(\text{pp}, x; r_i))$. This implies

$$\begin{aligned} (y^t, \pi^t) &= \text{Eval}^{f^t}(\text{pp}, x; r_E) = \text{Eval}^{h^{t'}}(\text{pp}, x; r_E) = (y''^{t'}, \pi''^{t'}), \\ (y_i^t, \pi_i^t) &= \text{Eval}^{g_i^t}(\text{pp}, x; r_i) = \text{Eval}^{f^{t'}}(\text{pp}, x; r_i) = (y^{t'}, \tilde{\pi}^{t'}), \text{ and} \\ \text{Verify}^{f^t}(\text{pp}, x, y^t, \pi^t) &= \text{Verify}^{h^{t'}}(\text{pp}, x, y''^{t'}, \pi''^{t'}). \end{aligned}$$

Here, equality between algorithms not only means that their outputs are equal, but also that they pass the exact same states during execution, and in particular make the same queries to their respective oracles. As $t \implies ((y'_i \neq y) \wedge E)$, these equalities immediately imply $t' \implies ((y''_i \neq y) \wedge E')$. The opposite implication can be proven similarly. $\square$

This concludes the proof of Theorem 4.

Acknowledgments

Hamza Abusalah is supported by the PICOCRYPT project that has received funding from the European Research Council (ERC) under the European Unions Horizon 2020 research and innovation programme (Grant agreement No. 101001283), partially supported by ESPADA (PID2022-142290OB-I00) and CEX2024-001471-M funded by MCIN/AEI/10.13039/501100011033/. Erkan Tairi is supported in part by the France 2030 ANR-22-PECY-003 SecureCompute Project, by the Austrian Science Fund (FWF) Project J4879-N, by the Bakar Funds and Peder Sather Funds.

References

- ABBK16. Frederik Armknecht, Ludovic Barman, Jens-Matthias Bohli, and Ghasan O. Karame. Mirror: Enabling proofs of data replication and retrievability in the cloud. In Thorsten Holz and Stefan Savage, editors, *USENIX Security 2016*, pages 1051–1068. USENIX Association, August 2016.
- Abu25. Hamza Abusalah. On graphs of incremental proofs of sequential work. In Tibor Jager and Jiaxin Pan, editors, *PKC 2025, Part V*, volume 15678 of *LNCS*, pages 203–233. Springer, Cham, May 2025.
- AC23. Hamza Abusalah and Valerio Cini. An incremental PoSW for general weight distributions. In Carmit Hazay and Martijn Stam, editors, *EUROCRYPT 2023, Part II*, volume 14005 of *LNCS*, pages 282–311. Springer, Cham, April 2023.
- AFGK22. Hamza Abusalah, Georg Fuchsbaauer, Peter Gazi, and Karen Klein. SNACKs: Leveraging proofs of sequential work for blockchain light clients. In Shweta Agrawal and Dongdai Lin, editors, *ASIACRYPT 2022, Part I*, volume 13791 of *LNCS*, pages 806–836. Springer, Cham, December 2022.
- AKK⁺19. Hamza Abusalah, Chethan Kamath, Karen Klein, Krzysztof Pietrzak, and Michael Walter. Reversible proofs of sequential work. In Yuval Ishai and Vincent Rijmen, editors, *EUROCRYPT 2019, Part II*, volume 11477 of *LNCS*, pages 277–291. Springer, Cham, May 2019.
- ARS24. Damiano Abram, Lawrence Roy, and Mark Simkin. Time-based cryptography from weaker assumptions: Randomness beacons, delay functions and more. Cryptology ePrint Archive, Report 2024/769, 2024.
- AS15. Joël Alwen and Vladimir Serbinenko. High parallel complexity graphs and memory-hard functions. In Rocco A. Servedio and Ronitt Rubinfeld, editors, *47th ACM STOC*, pages 595–603. ACM Press, June 2015.
- BBBF18. Dan Boneh, Joseph Bonneau, Benedikt Bünz, and Ben Fisch. Verifiable delay functions. In Hovav Shacham and Alexandra Boldyreva, editors, *CRYPTO 2018, Part I*, volume 10991 of *LNCS*, pages 757–788. Springer, Cham, August 2018.
- BCH⁺22. Nir Bitansky, Arka Rai Choudhuri, Justin Holmgren, Chethan Kamath, Alex Lombardi, Omer Paneth, and Ron D. Rothblum. PPAD is as hard as LWE and iterated squaring. In Eike Kiltz and Vinod Vaikuntanathan, editors, *TCC 2022, Part II*, volume 13748 of *LNCS*, pages 593–622. Springer, Cham, November 2022.
- BHR⁺21. Alexander R. Block, Justin Holmgren, Alon Rosen, Ron D. Rothblum, and Pratik Soni. Time- and space-efficient arguments from groups of unknown order. In Tal Malkin and Chris Peikert, editors, *CRYPTO 2021, Part IV*, volume 12828 of *LNCS*, pages 123–152, Virtual Event, August 2021. Springer, Cham.
- BR93. Mihir Bellare and Phillip Rogaway. Random oracles are practical: A paradigm for designing efficient protocols. In Dorothy E. Denning, Raymond Pyle, Ravi Ganesan, Ravi S. Sandhu, and Victoria Ashby, editors, *ACM CCS 93*, pages 62–73. ACM Press, November 1993.
- CLM23. Valerio Cini, Russell W. F. Lai, and Giulio Malavolta. Lattice-based succinct arguments from vanishing polynomials - (extended abstract). In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part II*, volume 14082 of *LNCS*, pages 72–105. Springer, Cham, August 2023.

- CP18. Bram Cohen and Krzysztof Pietrzak. Simple proofs of sequential work. In Jesper Buus Nielsen and Vincent Rijmen, editors, *EUROCRYPT 2018, Part II*, volume 10821 of *LNCS*, pages 451–467. Springer, Cham, April / May 2018.
- CP19. Bram Cohen and Krzysztof Pietrzak. The chia network blockchain, 2019. Last accessed on August 28, 2025 at <https://www.chia.net/wp-content/uploads/2022/07/ChiaGreenPaper.pdf>.
- CSRT22. Jorge Chávez-Saab, Francisco Rodríguez-Henríquez, and Mehdi Tibouchi. Verifiable isogeny walks: Towards an isogeny-based postquantum VDF. In Riham AlTawy and Andreas Hülsing, editors, *SAC 2021*, volume 13203 of *LNCS*, pages 441–460. Springer, Cham, September / October 2022.
- DGMV20. Nico Döttling, Sanjam Garg, Giulio Malavolta, and Prashant Nalini Vasudevan. Tight verifiable delay functions. In Clemente Galdi and Vladimir Kolesnikov, editors, *SCN 20*, volume 12238 of *LNCS*, pages 65–84. Springer, Cham, September 2020.
- DLM19. Nico Döttling, Russell W. F. Lai, and Giulio Malavolta. Incremental proofs of sequential work. In Yuval Ishai and Vincent Rijmen, editors, *EUROCRYPT 2019, Part II*, volume 11477 of *LNCS*, pages 292–323. Springer, Cham, May 2019.
- DMPS19. Luca De Feo, Simon Masson, Christophe Petit, and Antonio Sanso. Verifiable delay functions from supersingular isogenies and pairings. In Steven D. Galbraith and Shihō Moriai, editors, *ASIACRYPT 2019, Part I*, volume 11921 of *LNCS*, pages 248–277. Springer, Cham, December 2019.
- EFKP20a. Naomi Ephraim, Cody Freitag, Ilan Komargodski, and Rafael Pass. Continuous verifiable delay functions. In Anne Canteaut and Yuval Ishai, editors, *EUROCRYPT 2020, Part III*, volume 12107 of *LNCS*, pages 125–154. Springer, Cham, May 2020.
- EFKP20b. Naomi Ephraim, Cody Freitag, Ilan Komargodski, and Rafael Pass. SPARKs: Succinct parallelizable arguments of knowledge. In Anne Canteaut and Yuval Ishai, editors, *EUROCRYPT 2020, Part I*, volume 12105 of *LNCS*, pages 707–737. Springer, Cham, May 2020.
- GRY25. Ziyi Guan, Artur Riazanov, and Weiqiang Yuan. Breaking verifiable delay functions in the random oracle model. In Yael Tauman Kalai and Seny F. Kamara, editors, *CRYPTO 2025, Part VII*, volume 16006 of *LNCS*, pages 161–191. Springer, Cham, August 2025.
- GS98. David M. Goldschlag and Stuart G. Stubblebine. Publicly verifiable lotteries: Applications of delaying functions. In Rafael Hirschfeld, editor, *FC’98*, volume 1465 of *LNCS*, pages 214–226. Springer, Berlin, Heidelberg, February 1998.
- HHK⁺22. Charlotte Hoffmann, Pavel Hubáček, Chethan Kamath, Karen Klein, and Krzysztof Pietrzak. Practical statistically-sound proofs of exponentiation in any group. In Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part II*, volume 13508 of *LNCS*, pages 370–399. Springer, Cham, August 2022.
- HHKK23. Charlotte Hoffmann, Pavel Hubáček, Chethan Kamath, and Tomáš Krnák. (Verifiable) delay functions from lucas sequences. In Guy N. Rothblum and Hoeteck Wee, editors, *TCC 2023, Part IV*, volume 14372 of *LNCS*, pages 336–362. Springer, Cham, November / December 2023.
- IR89. Russell Impagliazzo and Steven Rudich. Limits on the provable consequences of one-way permutations. In *21st ACM STOC*, pages 44–61. ACM Press, May 1989.

- JMRR21. Samuel Jaques, Hart Montgomery, Razvan Rosie, and Arnab Roy. Time-release cryptography from minimal circuit assumptions. In Avishek Adhikari, Ralf Küsters, and Bart Preneel, editors, *INDOCRYPT 2021*, volume 13143 of *LNCS*, pages 584–606. Springer, Cham, December 2021.
- LM23. Russell W. F. Lai and Giulio Malavolta. Lattice-based timed cryptography. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part V*, volume 14085 of *LNCS*, pages 782–804. Springer, Cham, August 2023.
- LSS20. Esteban Landerreche, Marc Stevens, and Christian Schaffner. Non-interactive cryptographic timestamping based on verifiable delay functions. In Joseph Bonneau and Nadia Heninger, editors, *FC 2020*, volume 12059 of *LNCS*, pages 541–558. Springer, Cham, February 2020.
- LV20. Alex Lombardi and Vinod Vaikuntanathan. Fiat-shamir for repeated squaring with applications to PPAD-hardness and VDFs. In Daniele Micciancio and Thomas Ristenpart, editors, *CRYPTO 2020, Part III*, volume 12172 of *LNCS*, pages 632–651. Springer, Cham, August 2020.
- LW15. Arjen K. Lenstra and Benjamin Wesolowski. A random zoo: sloth, unicorn, and trx. Cryptology ePrint Archive, Report 2015/366, 2015.
- MMV11. Mohammad Mahmoody, Tal Moran, and Salil P. Vadhan. Time-lock puzzles in the random oracle model. In Phillip Rogaway, editor, *CRYPTO 2011*, volume 6841 of *LNCS*, pages 39–50. Springer, Berlin, Heidelberg, August 2011.
- MMV13. Mohammad Mahmoody, Tal Moran, and Salil P. Vadhan. Publicly verifiable proofs of sequential work. In Robert D. Kleinberg, editor, *ITCS 2013*, pages 373–388. ACM, January 2013.
- MSW20. Mohammad Mahmoody, Caleb Smith, and David J. Wu. Can verifiable delay functions be based on random oracles? In Artur Czumaj, Anuj Dawar, and Emanuela Merelli, editors, *ICALP 2020*, volume 168 of *LIPIcs*, pages 83:1–83:17. Schloss Dagstuhl, July 2020.
- OKC⁺25. Michal Osadnik, Darya Kaviani, Valerio Cini, Russell W. F. Lai, and Giulio Malavolta. Papercraft: Lattice-based verifiable delay function implemented. In Marina Blanton, William Enck, and Cristina Nita-Rotaru, editors, *2025 IEEE Symposium on Security and Privacy*, pages 1603–1621. IEEE Computer Society Press, May 2025.
- Pie19. Krzysztof Pietrzak. Simple verifiable delay functions. In Avrim Blum, editor, *ITCS 2019*, volume 124, pages 60:1–60:15. LIPIcs, January 2019.
- RSS20. Lior Rotem, Gil Segev, and Ido Shahaf. Generic-group delay functions require hidden-order groups. In Anne Canteaut and Yuval Ishai, editors, *EUROCRYPT 2020, Part III*, volume 12107 of *LNCS*, pages 155–180. Springer, Cham, May 2020.
- RSW96. R. L. Rivest, A. Shamir, and D. A. Wagner. Time-lock puzzles and timed-release crypto. Technical report, USA, 1996.
- Rud88. Steven Rudich. Limits on the provable consequences of one-way functions. Technical report, USA, 1988.
- Sta20. StarkWare. Veedo, 2020. Last accessed on August 28, 2025 at <https://medium.com/starkware/presenting-veedo-e4bbff77c7ae>.
- Wes19. Benjamin Wesolowski. Efficient verifiable delay functions. In Yuval Ishai and Vincent Rijmen, editors, *EUROCRYPT 2019, Part III*, volume 11478 of *LNCS*, pages 379–407. Springer, Cham, May 2019.